

ADDA Neural Preconditioner

техническая книга проекта

Состояние рабочей копии: convsai-universal

1 июня 2026

Оглавление

Зачем нужен этот документ	iv
1 Короткая картина проекта	1
1.1 Что мы ускоряем	1
1.2 Что такое нейросетевой предобуславливатель	1
1.3 Две ветки	1
2 Численная основа	3
2.1 DDA и матричный оператор	3
2.2 Почему FFT важен	3
2.3 Параметры grid, dpl, kd и m	3
3 Предобуславливание в ADDA	5
3.1 Левое предобуславливание	5
3.2 Почему предобуславливатель может замедлять	5
3.3 Wall time, iterations и residual	5
4 Архитектура ConvSAI/K2	7
4.1 Назначение	7
4.2 Входы модели	7
4.3 ShapeEncoder3D	7
4.4 MLP и stencil	8
4.5 Что значит K2	8
4.6 Сильные и слабые стороны	8
5 Архитектура Spectral ConvSAI	9
5.1 Главная идея	9
5.2 Вход на каждой частоте	9
5.3 Почему spectral лучше масштабируется	9
5.4 Transformer в spectral-модели	10
5.5 Correction branch	10
6 Экспорт в ADDA	11
6.1 Почему нужен export	11
6.2 ConvSAI/K2 export	11
6.3 Spectral export	11
6.4 Правило совпадения параметров	12
7 Форматы precond-файлов	13
7.1 Поддерживаемые modes	13
7.2 Mode 3	13
7.3 Mode 4	13
7.4 Mode 5	14

8 Сборка и запуск ADDA	15
8.1 Сборка patched ADDA	15
8.2 Sequential run	15
8.3 MPI run	15
8.4 FGMRES	16
9 Обучение	17
9.1 Файл trainer	17
9.2 Точность вычислений	17
9.3 Loss-функции	17
9.3.1 Probe loss	17
9.3.2 Adversarial probe loss	18
9.3.3 Planewave Krylov loss	18
9.3.4 Planewave BiCGStab loss	18
9.3.5 Зачем anchor-термы	18
10 Guarded validation	19
10.1 Зачем gate	19
10.2 Исправленная проблема ConvSAI/K2 validation	19
10.3 Фильтрация validation по форме	19
10.4 Как читать gate	20
11 Практические команды обучения	21
11.1 ConvSAI/K2 baseline training	21
11.2 ConvSAI/K2 large-grid gated fine-tune with anchors	21
11.3 Spectral continuation loop	22
12 Как смотреть логи	24
12.1 Процессы	24
12.2 GPU	24
12.3 Лог обучения	24
12.4 Файлы результата	24
12.5 Типичная строка training log	24
13 Heatmap и benchmark	25
13.1 Что должна показывать heatmap	25
13.2 Три разных speedup	25
13.3 Ориентации	25
14 Текущие результаты и ограничения	27
14.1 Spectral	27
14.2 ConvSAI/K2	27
15 Почему последние попытки были подозрительными	29
15.1 Проблема exporter в validation	29
15.2 Проблема нулевых anchors	29
15.3 Что значит “чинить” в этом месте	29
16 Ограничения и направления развития	30
16.1 Low-rank	30
16.2 Deflation	30
16.3 SO(3) equivariance	30
16.4 Analytic circulant inverse	30

16.5 Аугментации	31
17 Практический чеклист	32
17.1 Перед запуском	32
17.2 Перед доверием к checkpoint	32
17.3 Если нет ускорения	32
18 Карта файлов	33
Итоговая модель понимания	34

Зачем нужен этот документ

В этом проекте смешаны три уровня сложности: численный метод DDA, модификация ADDA на C и нейросетевое обучение предобуславливателя. Если смотреть только на логи обучения, легко сделать неверный вывод: *loss стал лучше, значит ADDA ускорится*. На практике это не так. Модель может уменьшать внутренний training loss и одновременно делать реальный расчёт медленнее.

Цель этой книги — дать не поверхностную карту файлов, а рабочую модель понимания:

- что именно решает ADDA;
- что такое предобуславливатель и почему он может ускорять или замедлять;
- чем отличаются ConvSAI/K2 и Spectral ConvSAI;
- как устроены входы, выходы, loss-функции и экспорт;
- как читать логи, heatmap и wall-time метрики;
- почему последние ConvSAI/K2 fine-tune попытки были подозрительными;
- какой порядок действий нужен, чтобы честно доказать полезность модели.

Глава 1

Короткая картина проекта

1.1. Что мы ускоряем

ADDA решает систему линейных уравнений, возникающую в DDA (Discrete Dipole Approximation):

$$Ax = b, \quad (1.1)$$

где A — большая комплексная матрица взаимодействия диполей, x — неизвестные дипольные поляризации, b — правая часть, задаваемая падающей электромагнитной волной. Матрица не формируется как плотная матрица; её применение реализуется через FFT.

Главная цена расчёта — итерационный решатель. В проекте чаще всего используется BiCGStab, а для некоторых больших preconditioner-heavy случаев тестируется FGMRES. Если без предобуславливателя нужно сотни или тысячи итераций, хороший предобуславливатель может уменьшить это число в разы или десятки раз.

1.2. Что такое нейросетевой предобуславливатель

Предобуславливатель — оператор M , который должен примерно вести себя как A^{-1} . Вместо исходной системы решатель работает с более удобной системой:

$$MAx = Mb. \quad (1.2)$$

Если $MA \approx I$, спектр системы становится лучше, и Krylov-решатель сходится быстрее. Но M сам стоит денег: его надо импортировать, хранить и применять на каждом шаге. Поэтому реальная цель:

$$T_{\text{precond}} < T_{\text{baseline}}, \quad (1.3)$$

а не просто меньшее число итераций или красивый loss.

1.3. Две ветки

Проект разделён на две независимые ветки.

conv sai-universal

Текущая рабочая копия. Содержит ConvSAI Universal, также

называемый K2 или K² v3. Это пространственный свёрточный предобуславливатель.

spectral-convsaï

Отдельная ветка со Spectral ConvSAI. Она предсказывает предобуславливатель прямо в частотной области и лучше подходит для больших grid, но требует problem-specific export.

Глава 2

Численная основа

2.1. DDA и матричный оператор

В DDA частица заменяется набором диполей. Для каждого диполя есть три компоненты поляризации, поэтому размер системы равен:

$$n = 3N, \quad (2.1)$$

где N — число диполей. Оператор обычно можно представить как:

$$A = I - \alpha G, \quad (2.2)$$

где α — поляризуемость, а G — оператор взаимодействия диполей. В коде Python эта операция реализована в `core/fft_matvec.py`. В ADDA она реализована в нативном C-коде.

2.2. Почему FFT важен

Наивное применение плотной матрицы A стоило бы $O(N^2)$. За счёт свёрточной структуры взаимодействия применение Av можно делать через FFT:

$$Av = v - \alpha \mathcal{F}^{-1} \left(\hat{G} \mathcal{F}(v) \right). \quad (2.3)$$

Это делает один `matvec` намного дешевле, но всё равно при сотнях итераций он остаётся главным временем расчёта.

2.3. Параметры `grid`, `dpl`, `kd` и `m`

Параметр	Значение в проекте
<code>grid</code>	Размер дискретизации формы. Например <code>grid=80</code> .
<code>dpl</code>	Dipoles per wavelength в ADDA. Часто используется <code>-dpl 15</code> .
<code>kd</code>	Параметр обучения Python. В этом проекте <code>kd = 2*pi/dpl</code> .
<code>m</code>	Комплексный показатель преломления: $m = m_{\text{re}} + im_{\text{im}}$.
<code>eps</code>	Точность ADDA. <code>-eps 3</code> соответствует порядку 10^{-3} .

Для `dpl=15`:

$$kd = \frac{2\pi}{15} = 0.41887902047863906. \quad (2.4)$$

Экспорт нейросетевого предобуславливателя и запуск ADDA должны использовать одинаковые `grid`, `shape`, `m` и `dpl/kd`. Если экспорт сделан под одну задачу, а ADDA запущена на другой, результат нельзя считать валидным.

Глава 3

Предобуславливание в ADDA

3.1. Левое предобуславливание

В патче ADDA для BiCGStab реализовано левое предобуславливание:

$$MAx = Mb. \quad (3.1)$$

Это значит, что в итерационном методе многие векторы живут в предобусловленном пространстве. В `adda_src_modified/iterative.c` есть отдельная логика для сохранения фактической невязки исходной системы, потому что внутренняя невязка Mr не равна физической невязке $r = b - Ax$.

3.2. Почему предобуславливатель может замедлять

Пусть `baseline` требует k_0 итераций, а `run` с предобуславливателем требует k_M . Пусть стоимость одной `baseline`-итерации равна C_A , а стоимость итерации с предобуславливателем равна $C_A + C_M$. Тогда грубая модель:

$$T_0 \approx k_0 C_A, \quad (3.2)$$

$$T_M \approx T_{\text{import}} + k_M(C_A + C_M). \quad (3.3)$$

Ускорение есть только если:

$$k_M(C_A + C_M) + T_{\text{import}} < k_0 C_A. \quad (3.4)$$

Поэтому недостаточно уменьшить число итераций на 20%, если каждая итерация стала в 2 раза дороже.

3.3. Wall time, iterations и residual

Нужно различать три метрики:

Iterations	Сколько итераций сделал решатель. Полезно, но не равно времени.
Final residual	К какой невязке пришёл решатель. Если оба <code>run</code> упёрлись в <code>maxiter</code> , <code>final residual</code> может быть важнее формального <code>speedup</code> .
Wall time	Реальное время по часам. Главная метрика для пользователя.

Если видим:

```
hex_DL1_g48: 300->300 (1.0x)
```

это означает, что baseline и preconditioned run оба упёрлись в лимит 300 итераций. Такой чекпоинт не должен становиться `best_model.pt`.

Глава 4

Архитектура ConvSAI/K2

4.1. Назначение

ConvSAI/K2 — универсальная пространственная модель. Она по форме частицы и физическим параметрам предсказывает локальное свёрточное ядро предобуславливателя. Главный класс:

```
neural_precond.model.ConvSAI_Universal
```

Основной checkpoint:

```
models/k2v3/checkpoints/best_model.pt
```

4.2. Входы модели

Модель получает:

- occupancy grid формы: бинарный тензор 0/1;
- m_{re} ;
- m_{im} ;
- kd ;
- $\log(\text{grid})$.

Функция `positions_to_occupancy` переводит позиции диполей в 3D occupancy:

$$\text{positions} \rightarrow X \in \{0, 1\}^{1 \times 1 \times G \times G \times G}. \quad (4.1)$$

4.3. ShapeEncoder3D

Геометрия кодируется небольшой 3D CNN:

```
Conv3d(1 -> 16), ReLU, MaxPool3d
Conv3d(16 -> 32), ReLU, MaxPool3d
Conv3d(32 -> 64), ReLU, MaxPool3d
AdaptiveAvgPool3d(1)
Linear(64 -> shape_embed_dim)
```

Перед encoder occupancy приводится к фиксированному разрешению `encoder_resolution`. Это позволяет подавать разные grid, но означает, что очень тонкие геометрические детали могут сглаживаться.

4.4. MLP и stencil

После encoder модель склеивает:

$$[m_{\text{re}}, m_{\text{im}}, kd, \log(g), s], \quad (4.2)$$

где s — shape embedding. При shape_embed_dim=16 вход MLP имеет 20 чисел. MLP выдаёт комплексное ядро:

$$K[\Delta x, \Delta y, \Delta z] \in \mathbb{C}^{3 \times 3}. \quad (4.3)$$

При r_cut=7 stencil содержит 1419 смещений. На каждом смещении хранится 3 на 3 комплексный блок.

4.5. Что значит K2

K2 или K^2 означает, что модель предсказывает ядро K , но применяется:

$$\widehat{M}(k) = \widehat{K}(k) \widehat{K}(k). \quad (4.4)$$

В пространстве это увеличивает эффективный радиус примерно с 7 до 14 клеток. Параметров модели не становится больше, но экспортируемый stencil после возведения в квадрат становится шире.

4.6. Сильные и слабые стороны

Сторона	Практический смысл
Плюс	Один checkpoint можно использовать для разных форм и параметров.
Плюс	Mode 3 файл обычно компактнее spectral-export с большим radius.
Плюс	Простая архитектура и быстрый forward.
Минус	Effective radius около 14 может быть недостаточен для больших grid.
Минус	Универсальность снижает специализацию под hard hex-prism cases.
Минус	Loss может улучшаться без улучшения real ADDA wall time.

Глава 5

Архитектура Spectral ConvSAI

5.1. Главная идея

Spectral ConvSAI работает не как локальный spatial stencil predictor, а как частотный predictor. Для каждой точки FFT-сетки модель предсказывает:

$$\widehat{M}(k) \in \mathbb{C}^{3 \times 3}. \quad (5.1)$$

Главный класс:

```
neural_precond.model.ConvSAI_Spectral
```

Ветка:

```
spectral-conv sai
```

5.2. Вход на каждой частоте

Для каждой частоты модель получает:

- полный комплексный $D_{\text{hat}}(k)$, разложенный в 18 вещественных чисел;
- координаты частоты;
- глобальный conditioning-вектор от формы и физических параметров.

Глобальный conditioning строится так же концептуально: shape encoder плюс $m_{\text{re}}, m_{\text{im}}, kd, \log(g)$.

5.3. Почему spectral лучше масштабируется

ConvSAI/K2 ограничен spatial radius. Spectral-модель сразу работает в частотной области, где operator inverse естественно описывается как функция частоты. Поэтому на больших grid и сложных refractive indices spectral-модель часто сильнее.

Цена:

- экспорт зависит от конкретной задачи;
- файл может быть больше;
- import и apply могут стать существенной частью wall time.

5.4. Transformer в spectral-модели

Transformer здесь не является моделью, которая напрямую выдаёт весь пре-добуславливатель. Он используется как coarse frequency context.

Если включить:

```
--spectral_transformer_tokens 8
```

то берётся грубая сетка частот:

$$i_x, i_y, i_z = \text{linspace}(0, g - 1, n_{\text{tokens}}). \quad (5.2)$$

Для выбранных точек собираются признаки D_{hat} , координаты частоты и квадратичные признаки. Transformer выдаёт поправку к global conditioning, после чего обычный per-frequency MLP предсказывает $\widehat{M}(k)$ на всех частотах.

5.5. Correction branch

В spectral-модели есть zero-initialized additive correction branch. Она нужна, чтобы дообучать небольшую поправку, не разрушая базовый checkpoint. Практически это безопаснее, чем размораживать всю модель, когда уже есть рабочая spectral версия.

Глава 6

Экспорт в ADDA

6.1. Почему нужен export

ADDA не исполняет PyTorch. Она читает бинарный файл .precond. Поэтому любая модель проходит цепочку:

$$\text{checkpoint} \rightarrow \text{export script} \rightarrow \text{.precond} \rightarrow \text{ADDA.} \quad (6.1)$$

Если export script не соответствует архитектуре checkpoint, результат либо не загрузится, либо будет математически неправильным.

6.2. ConvSAI/K2 export

```
python3 apps/export_universal_precond.py \  
  --checkpoint models/k2v3/checkpoints/best_model.pt \  
  --squared_kernel \  
  --shape prism \  
  --ay 6.0 \  
  --az 1.0 \  
  --grid 80 \  
  --m_re 2.5 \  
  --m_im 0.0 \  
  --kd 0.41887902047863906 \  
  --output exports/prism_g80_m25_k2v3.precond
```

Здесь --squared_kernel обязателен для K2 checkpoint, иначе экспорт будет соответствовать другой математике.

6.3. Spectral export

```
python3 apps/export_spectral_precond.py \  
  --checkpoint models/spectral/checkpoints/  
    best_hex_prism_real_32to96_r40_quality10h_sym.pt \  
  --shape prism \  
  --ay 6.0 \  
  --az 1.0 \  
  --grid 80 \  
  --m_re 2.5 \  
  --m_im 0.0 \  
  --kd 0.41887902047863906 \  

```

```
--threshold-rel 1e-6 \
--max-radius 40 \
--blend-identity 1.0 \
--symmetry z180_zflip \
--output exports/prism_g80_m25_spectral.precond
```

Параметры `--threshold-rel`, `--max-radius` и `--symmetry` контролируют, сколько spatial-компонент будет сохранено после преобразования из частотного представления.

6.4. Правило совпадения параметров

Export	ADDA
--grid 80	-grid 80
--shape prism --ay 6	-shape prism 6.0 1.0
--az 1	
--m_re 2.5 --m_im 0.0	-m 2.5 0.0
--kd 0.41887902047863906	-dpl 15

Несовпадение этих параметров является одной из самых частых причин “ускорения без ускорения”.

Глава 7

Форматы precondition-файлов

7.1. Поддерживаемые modes

В `adda_src_modified/precond.h` определены основные modes:

Mode	Что хранит
0	ILU factors.
1	Sparse approximate inverse matrix.
2	Polynomial preconditioner.
3	ConvSAI spatial stencil.
4	FFTDIRECT: готовый frequency-domain kernel.
5	FFTDIRECT_XSLAB_F32: x-slab float32 layout для MPI.

7.2. Mode 3

Mode 3 хранит spatial stencil:

$$\{(\Delta x_s, \Delta y_s, \Delta z_s), K_s\}_{s=1}^S. \quad (7.1)$$

При импорте ADDA строит frequency-domain kernel через FFT. Это компактно, но на больших stencil и MPI import может быть заметным.

7.3. Mode 4

Mode 4 хранит уже готовый $\widehat{M}$ на FFT-сетке. Import быстрее, потому что ADDA не надо пересобирать $\widehat{M}$ из spatial stencil.

```
python3 apps/convert_conv sai_to_fftdirect.py \  
--input exports/prism_g80_m25_spectral.precond \  
--output exports/prism_g80_m25_spectral.fftdirect.precond \  
--grid-x 160 \  
--grid-y 192 \  
--grid-z 160
```

Важно: grid-x/y/z — это FFT grid из ADDA log, а не просто particle -grid.

7.4. Mode 5

Mode 5 хранит frequency kernel в float32 по x-slab. Это полезно для MPI, потому что каждый процесс может читать свою часть, а не весь dense kernel.

```
python3 apps/convert_fftdirect_to_xslab_f32.py \  
--input exports/prism_g80_m25_spectral.fftdirect.precond \  
--output exports/prism_g80_m25_spectral.xslab_f32.precond
```

Mode 4/5 ускоряют import и уменьшают overhead. Они не исправляют плохой математический предобуславливатель.

Глава 8

Сборка и запуск ADDA

8.1. Сборка patched ADDA

Репозиторий не хранит весь upstream ADDA. Нужно взять чистую ADDA и наложить патч:

```
git clone https://github.com/adda-team/adda adda
cp adda_src_modified/* adda/src/

make -C adda/src seq mpi \
  FFTW3_INC_PATH="$HOME/.local/include" \
  FFTW3_LIB_PATH="$HOME/.local/lib"
```

8.2. Sequential run

```
export LD_LIBRARY_PATH="$HOME/.local/lib:${LD_LIBRARY_PATH:-}"

adda/src/seq/adda \
  -dir runs/prism_g80_m25_seq \
  -grid 80 \
  -m 2.5 0.0 \
  -shape prism 6.0 1.0 \
  -dpl 15 \
  -eps 3 \
  -iter bicgstab \
  -precond exports/prism_g80_m25_spectral.precond
```

8.3. MPI run

```
export LD_LIBRARY_PATH="$HOME/.local/lib:${LD_LIBRARY_PATH:-}"

mpirun -np 16 adda/src/mpi/adda_mpi \
  -dir runs/prism_g80_m25_mpi \
  -grid 80 \
  -m 2.5 0.0 \
  -shape prism 6.0 1.0 \
  -dpl 15 \
  -eps 3 \
  -iter bicgstab \
```

```
-precond exports/prism_g80_m25_spectral.precond
```

8.4. FGMRES

Для больших FFTDIRECT cases иногда полезен FGMRES:

```
ADDA_FGMRES_RESTART=100 \  
mpirun -np 16 adda/src/mpi/adda_mpi \  
-dir runs/prism_g80_m25_mpi_fgmres \  
-grid 80 \  
-m 2.5 0.0 \  
-shape prism 6.0 1.0 \  
-dpl 15 \  
-eps 3 \  
-iter fgmres \  
-precond exports/prism_g80_m25_spectral.precond
```

FGMRES использует правое предобуславливание и может требовать меньше preconditioner applications на эффективный шаг, но его надо сравнивать эмпирически на целевых cases.

Глава 9

Обучение

9.1. Файл `trainer`

Главный `trainer`:

```
train_v7/train.py
```

Он делает цикл:

1. сэмплирует форму, `grid`, m , kd ;
2. строит позиции диполей;
3. строит `FFTMatVec`;
4. строит `occupancy grid`;
5. получает `kernel` или `spectral conditioning` от модели;
6. считает `loss`;
7. делает `backward` и `AdamW step`;
8. периодически запускает `validation`.

Базовый `seed` в `trainer`:

```
seed = 42
```

9.2. Точность вычислений

По умолчанию веса модели обучаются в `fp32`. Однако многие операции с системой, `FFTMatVec` и комплексными векторами выполняются в `complex128`. `Export` пишет комплексные `double` значения в `.precond`. `AMP/fp16` в стандартной команде обучения не включены.

9.3. Loss-функции

9.3.1. Probe loss

Идея:

$$\mathcal{L}_{\text{probe}} = \mathbb{E}_z \frac{\|MAz - z\|^2}{\|z\|^2}. \quad (9.1)$$

Это проверяет, похож ли M на левый inverse. Минус: случайные векторы могут плохо коррелировать с реальными plane-wave RHS в ADDA.

9.3.2. Adversarial probe loss

Ищет плохие направления для $I - MA$ через power iteration, затем учит модель на них. Это сильнее обычного probe loss, но всё равно не гарантирует real ADDA speedup.

9.3.3. Planewave Krylov loss

Использует RHS, похожие на ADDA plane wave. Вместо случайной правой части строятся две поляризации падающей волны. Это ближе к реальной задаче.

9.3.4. Planewave BiCGStab loss

Unrolled left-preconditioned BiCGStab loss. Он пытается оптимизировать именно то, что делает ADDA solver. Но у него есть опасность: если loss слишком свободный, модель может улучшать краткий unroll, не оставаясь хорошим приближением A^{-1} .

9.3.5. Зачем anchor-термы

Anchor-термы возвращают модель к смыслу inverse:

$$MAz \approx z, \quad (9.2)$$

$$AMz \approx z. \quad (9.3)$$

Практический вывод: для ConvSAI/K2 fine-tune на planewave_bicgstab опасно ставить:

```
--anchor_probe_weight 0.0
--anchor_right_probe_weight 0.0
```

Такой запуск может показывать улучшающийся loss, но не давать ускорения в ADDA. Более разумный старт:

```
--anchor_probe_weight 0.1
--anchor_right_probe_weight 0.1
--anchor_num_probes 1
--anchor_probe_chunk 1
```

Глава 10

Guarded validation

10.1. Зачем gate

Нельзя сохранять `best_model.pt` только потому, что `training loss` стал лучше. Нужна схема:

train chunk → export candidate → ADDA MPI validation → promote only if better. (10.1)

Именно для этого используется:

```
--guarded_val  
--solve_val_interval 250  
--guarded_min_large_speedup 1.01
```

10.2. Исправленная проблема ConvSAI/K2 validation

В текущей рабочей копии исправлена важная ошибка: large-grid ADDA validation раньше был жёстко завязан на `spectral exporter`. Для ConvSAI/K2 это неправильно.

Правильная логика:

```
if spectral:  
    apps/export_spectral_precond.py  
else:  
    apps/export_universal_precond.py
```

Также для SquaredConvSAI validation checkpoint сохраняется без префикса `base.`, чтобы `export_universal_precond.py` мог его загрузить.

10.3. Фильтрация validation по форме

Если обучение запущено так:

```
--only_shape hex_prism  
--hex_dl_min 1.0  
--hex_dl_max 1.0
```

то validation должен проверять hex-prism DL=1, а не sphere. Поэтому добавлена фильтрация fixed validation configs по `only_shape` и диапазону DL.

10.4. Как читать gate

Пример плохого, но корректно обработанного результата:

```
Validation (3 configs) speedup: 1.12x  
hex_DL1_g32: 1000->751 (1.3x) | hex_DL1_g40: 1000->1000 (1.0x) |  
  hex_DL1_g48: 300->300 (1.0x)  
Guarded best gate: FAIL; required large-grid ADDA cases did not pass:  
  hex_DL1_g48  
>>> Best update skipped: guarded large-grid gate failed
```

Это значит, что `trainer` не испортил `best_model.pt`. `Periodic model_step500.pt` может существовать, но он не является хорошей моделью.

Глава 11

Практические команды обучения

11.1. ConvSAI/K2 baseline training

```
python3 train_v7/train.py \  
  --name convsai_universal_k2_v3 \  
  --device 0 \  
  --save \  
  --loss adversarial \  
  --squared_kernel \  
  --r_cut 7 \  
  --hidden_size 512 \  
  --num_layers 4 \  
  --grid_min 8 \  
  --grid_max 64 \  
  --m_re_min 1.5 \  
  --m_re_max 4.0 \  
  --kd_min 0.2 \  
  --kd_max 0.8 \  
  --ema_decay 0.999 \  
  --warmup_steps 500 \  
  --num_steps 60000 \  
  --lr 5e-4
```

11.2. ConvSAI/K2 large-grid gated fine-tune with anchors

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \  
python3 -u train_v7/train.py \  
  --resume models/k2v3/checkpoints/best_model.pt \  
  --name convsai_k2_hex_g32to96_adda_gated_anchor \  
  --device 0 \  
  --save \  
  --loss planewave_bicgstab \  
  --squared_kernel \  
  --r_cut 7 \  
  --hidden_size 512 \  
  --num_layers 4 \  
  --only_shape hex_prism \  
  --hex_dl_min 1.0 \  
  --hex_dl_max 1.5
```

```

--hex_dl_max 1.0 \
--grid_min 32 \
--grid_max 96 \
--m_re_min 2.0 \
--m_re_max 3.2 \
--m_im_min 0.0 \
--m_im_max 0.0 \
--kd_min 0.41887902047863906 \
--kd_max 0.41887902047863906 \
--krylov_iters 1 \
--anchor_probe_weight 0.1 \
--anchor_right_probe_weight 0.1 \
--anchor_num_probes 1 \
--anchor_probe_chunk 1 \
--lr 2e-6 \
--warmup_steps 100 \
--ema_decay 0.0 \
--curriculum_frac 0.0 \
--guarded_val \
--guarded_min_large_speedup 1.01 \
--solve_val_interval 250 \
--val_interval 0 \
--adda_val_np 16 \
--adda_val_eps 3 \
--adda_val_maxiter_base 300 \
--adda_val_maxiter_precond 300 \
--adda_val_timeout 600 \
--adda_mpi_bin adda/src/mpi/adda_mpi \
--fftw_lib_path "$HOME/.local/lib" \
--skip_final_eval \
--log_interval 10 \
--save_interval 250 \
--num_steps 5000

```

11.3. Spectral continuation loop

В ветке `spectral-conv sai` используется `promotion loop`:

```

DEVICE=0 \
SEED_BASE=6200 \
CYCLES=200 \
STEPS_PER_CYCLE=25 \
LR=2e-6 \
TRAIN_GRID_MIN=32 \
TRAIN_GRID_MAX=96 \
VAL_GRIDS=80,96 \
SCORE_MODE=max \
M_RE=3.0 \
M_IM=0.0 \
DPL=15 \
KD=0.41887902047863906 \
RADIUS=40 \
LOSS=planewave_bicgstab \
KRYLOV_ITERS=1 \
ANCHOR_PROBE_WEIGHT=0.0 \
ANCHOR_RIGHT_PROBE_WEIGHT=0.0 \

```

```
SPECTRAL_FREQ_CHUNK_SIZE=65536 \  
SPECTRAL_FREQ_CHECKPOINT_CHUNKS=1 \  
CURRICULUM_FRAC=0.0 \  
NP=16 \  
VAL_MAXITER=120 \  
VAL_TIMEOUT=600 \  
EXPORT_SYMMETRY=z180_zflip \  
START_CHECKPOINT=models/spectral/checkpoints/  
    best_hex_prism_real_32to96_r40_quality10h_sym.pt \  
BEST_CHECKPOINT=models/spectral/checkpoints/  
    best_hex_prism_real_32to96_r40_quality10h_sym_resume.pt \  
BEST_SCORE_FILE=models/spectral/checkpoints/  
    best_hex_prism_real_32to96_r40_quality10h_sym_resume.score \  
NAME_PREFIX=SPECTRAL_RESUME_G32T096_SYM_R40 \  
RUN_ROOT=runs/SPECTRAL_RESUME_G32T096_SYM_R40 \  
./train_spectral_hex_real_loop.sh
```

Глава 12

Как смотреть логи

12.1. Процессы

```
ps -eo pid,ppid,stat,etime,cmd | rg 'train_v7|adda_mpi|mpirun'
```

12.2. GPU

```
nvidia-smi
```

12.3. Лог обучения

```
tail -f logs/<RUN_NAME>/train.log
```

12.4. Файлы результата

```
find results/<RUN_NAME> -maxdepth 1 -type f -printf '%f %s\n' | sort
```

12.5. Типичная строка training log

```
[500/5000] loss=-1.205630 gnorm=10.9315 lr=2.0e-06 1.4 steps/s (hex_DL1  
.00 g48 m=2.70+0.00i kd=0.42)
```

loss	Внутренний training loss. Не равен speedup.
gnorm	Норма градиента после clipping.
steps/s	Скорость обучения. На больших grid она резко ниже.
hex_DL1.00 g48	Текущая сэмплированная задача.

Глава 13

Heatmap и benchmark

13.1. Что должна показывать heatmap

Хорошая heatmap должна измерять не только факт запуска, а минимум:

- baseline wall time;
- preconditioned wall time;
- speedup;
- iterations;
- final residual;
- converged/capped status;
- export/import overhead, если это часть workflow.

13.2. Три разных speedup

heatmap_adda_wall_speedup

Ускорение самого ADDA solve.

heatmap_precond_wall_s

Абсолютное время preconditioned run.

heatmap_total_elapsed_speedup

Ускорение полного pipeline, если туда входят export/import.

Для одного запуска total elapsed speedup может быть плохим, потому что export и import не успевают окупиться. Для `-orient avg` на сотни ориентаций import/export amortized, и картина может быть лучше.

13.3. Ориентации

Ускорение может зависеть от orientation, потому что меняется правая часть, путь сходимости и взаимодействие с симметрией частицы. Для честной оценки `-orient avg` нужно тестировать не один угол, а набор ориентаций:

$$S = \frac{1}{N} \sum_{i=1}^N \frac{T_{0,i}}{T_{M,i}}. \quad (13.1)$$

Также стоит смотреть worst case:

$$S_{\min} = \min_i \frac{T_{0,i}}{T_{M,i}}. \quad (13.2)$$

Глава 14

Текущие результаты и ограничения

14.1. Spectral

По актуальным заметкам ветки `spectral-conv sai`:

mean ADDA wall-time speedup: 9.93x
max ADDA wall-time speedup: 23.33x
mean total elapsed speedup including export: 4.43x
max total elapsed speedup including export: 15.87x

После MPI distributed fix были локальные проверки:

Case	Baseline	New precondition	Speedup
sphere g32, m=3	15.31 s	2.13 s	7.19x
sphere g48, m=3	108.26 s	8.09 s	13.38x
sphere g64, m=2	141.24 s	53.69 s	2.63x

Вывод: 20x возможно в отдельных областях heatmap, но не является постоянной гарантией по всем grid, m, shape и orientation.

14.2. ConvSAI/K2

Исторически K2 v3 давал большие итерационные ускорения на умеренных задачах, например около 30x по итерациям на некоторых g33 cases. Но свежий hard hex-prism test показал плохую картину:

Run	Wall time	Iters	Status
baseline	21.291 s	600	cap
k2 original	43.588 s	600	cap, worse
k2 step1500	42.843 s	600	residual
k2 step2000	43.496 s	600	cap, worse
spectral	38.101 s	542	residual
			partial convergence

Это не значит, что ConvSAI/K2 бесполезен вообще. Это значит, что на hard large-grid prism case текущая K2 модель не даёт нужного ускорения и не должна дообучаться вслепую.

Глава 15

Почему последние попытки были подозрительными

15.1. Проблема exporter в validation

Если ConvSAI/K2 validation использует spectral exporter, то real ADDA gate не проверяет настоящую ConvSAI/K2 модель. Это исправлено в train_v7/train.py.

15.2. Проблема нулевых anchors

Запуск с:

```
--anchor_probe_weight 0.0  
--anchor_right_probe_weight 0.0
```

слишком свободен. Он может оптимизировать короткий unrolled BiCGStab path, но не удерживать M как приблизительный inverse. Симптом совпал с наблюдением: loss улучшается, а large-grid ADDA gate остаётся 300->300.

15.3. Что значит “чинить” в этом месте

Практически правильный фикс:

1. не продолжать blind run;
2. запускать с anchor weights;
3. оставить guarded_val;
4. сохранять best_model.pt только после ADDA PASS;
5. если на 750-1000 шагах gate стабильно 300->300, менять подход, а не просто ждать 10 часов.

Глава 16

Ограничения и направления развития

16.1. Low-rank

Low-rank блок добавляет несколько глобальных мод:

$$M \approx M_{\text{local}} + UCV^*, \quad (16.1)$$

где локальная часть ловит ближнюю физику, а low-rank часть ловит дальние медленные modes. Это естественное развитие для ConvSAI/K2, потому что его главная слабость — ограниченный radius.

16.2. Deflation

Deflation явно убирает из Krylov-решателя плохие медленно сходящиеся направления. Идея:

$$\mathcal{K} \rightarrow \mathcal{K} \oplus \text{span}(u_1, \dots, u_r). \quad (16.2)$$

Это может быть полезно, если heatmap показывает повторяющиеся bad modes.

16.3. SO(3) equivariance

Обычный 3D CNN не гарантирует корректного поведения при произвольных поворотах. Для -orient avg полезна архитектура или аугментация, которая учитывает вращения:

$$f(Rx) = Rf(x)R^{-1}. \quad (16.3)$$

Для призм также важны дискретные симметрии, например z180 и zflip.

16.4. Analytic circulant inverse

Так как FFT-оператор уже имеет block-circulant структуру, можно строить часть предобуславливателя аналитически в частотной области, а нейросетью учить только поправку. Spectral branch ближе всего к этой идее.

16.5. Аугментации

Нужные аугментации для ориентаций:

- случайные alpha beta gamma;
- повороты occupancy grid;
- symmetry-aware export для призмы;
- validation на нескольких orientation seeds;
- отдельный average и worst-case speedup.

Глава 17

Практический чеклист

17.1. Перед запуском

1. Убедиться, что ветка правильная: `convsai-universal` или `spectral-convsai`.
2. Собрать patched ADDA.
3. Проверить `LD_LIBRARY_PATH`.
4. Проверить, что `export` параметры совпадают с ADDA параметрами.
5. Проверить, что используется правильный `exporter`.

17.2. Перед доверием к `checkpoint`

1. Это `best_model.pt`, а не случайный `model_step*.pt`.
2. В логе есть `Guarded best gate: PASS`.
3. Есть сравнение с `baseline` на той же задаче.
4. Проверены `wall time`, `iterations` и `final residual`.
5. Проверено несколько `grid/m/orientation`.

17.3. Если нет ускорения

1. Проверить совпадение `export/run` параметров.
2. Проверить, не упёрлись ли оба `run` в `maxiter`.
3. Проверить `final residual`.
4. Проверить `preconditioner apply time`.
5. Проверить `import overhead`.
6. Для `ConvSAI/K2` проверить `anchors` и `guarded validation`.
7. Для `Spectral` проверить `radius`, `threshold` и `symmetry`.

Глава 18

Карта файлов

Файл или папка	Назначение
README.md	Краткая инструкция текущей ветки.
docs/PROJECT_BOOK_RU.tex	Этот LaTeX-документ.
docs/PROJECT_BOOK_RU.md	Старый Markdown-конспект, оставлен как черновик.
train_v7/train.py	Trainer, validation, checkpoint logic.
neural_precond/model.py	ConvSAI, Spectral, ShapeEncoder, transformer.
neural_precond/loss.py	Loss-функции.
apps/export_universal_precond.py	ConvSAI/K2 export.
apps/export_spectral_precond.py	Spectral export в ветке spectral-conv sai.
apps/convert_conv sai_to_fftdirect.py	Mode 3 > mode 4.
apps/convert_fftdirect_to_xst.py	Mode 4 > mode 5.
adda_src_modified/precond.c	Загрузка и применение preconditioner в ADDA.
adda_src_modified/iterative.c	Изменения solver для preconditioning.
models/k2v3/checkpoints/best_conv_sai_k2_checkpoint	Best ConvSAI/K2 checkpoint.

Итоговая модель понимания

Если сжать весь проект до десяти пунктов:

1. ADDA решает $Ax = b$ итерационным методом.
2. Предобуславливатель M должен делать MA ближе к identity.
3. Реальная метрика — wall time, а не training loss.
4. ConvSAI/K2 — универсальный local spatial stencil predictor.
5. Spectral — frequency-domain predictor, лучше подходящий для больших grid.
6. Export должен точно соответствовать shape/grid/m/kd.
7. Mode 4/5 ускоряют import, но не чинят плохой M .
8. Checkpoint должен проходить real ADDA gate.
9. Для -orient avg нужно тестировать много ориентаций.
10. Для цели 20x основной кандидат сейчас — Spectral branch; ConvSAI/K2 требует осторожного gated fine-tune или архитектурного усиления.